Manual de datos

La tabla `catalogo_completo`

Una fila = **un visionado** (si una película se vio 3 veces, hay 3 filas). Vive en una hoja de Google, publicada como CSV para que el sitio la lea.

#	Columna	Tipo	Quién la escribe	Descripción
A	`id`	texto	histórico: script · bot: fórmula	Identificador. Histórico: `P0001-1` (nº + sufijo de revisión). Bot: `="P"&año&"-"&nº` por fórmula.
B	`numero`	número	histórico: script · bot: fórmula	Orden de visionado dentro del año (`anio_visto`). Bot: `COUNTIF` por-fila.
C	`titulo`	texto	usuario / IA	Título. El bot lo corrige (typos) con Gemini antes de guardar.
D	`director`	texto	IA	Lo completa Gemini a partir del título.
E	`anio_estreno`	número	IA	Año de estreno de la película.
F	`pais_origen`	texto	IA	País o países de origen (`USA`, `Argentina`, `Francia / Alemania`…).
G	`fecha_vista`	fecha	usuario	Día en que se vio. **Dato cargado a mano.**
H	`anio_visto`	número	usuario	Año de la lista anual (una pestaña por año en la planilla original).
I	`es_revisionado`	texto (`Sí` / `No`)	histórico: script · bot: fórmula	Si el título aparece más de una vez en el catálogo.
J	`estado_enriquecimiento`	select	flujo	`Migrada` / `Aprobada` (históricos) · `Enriquecida IA` (las que agrega el bot).
K	`fuente`	select	flujo	`CSV historico` · `Bot Telegram`.
L	`genero`	multi-select	IA	2-3 géneros. Solo poblado para las que agregó el bot (las históricas toman género de TMDB, ver abajo).
M	`animo`	multi-select	IA	Reservado (tensa, feel-good, lenta…). Sin uso todavía.
N	`sinopsis`	texto largo	IA	Sinopsis breve (2 frases).
O	`poster_url`	URL	reservado	Reservado. Hoy el póster viene de `posters.json`, no de acá.
P	`rating_externo`	número	reservado	Reservado. Idem rating.
Q	`notas_ia`	texto largo	flujo	Lo que propuso la IA, para revisar. Sin uso todavía.

#	Columna	Tipo	Quién la escribe	Descripción
R	`revisionado`	fórmula	hoja	`=ARRAYFORMULA(…COUNTIF…)` — recuenta revisiones sobre toda la columna C.
S	`numero_orden`	fórmula	hoja	Numeración correlativa por año, calculada.

> **Por qué R y S son fórmulas aparte de I y B:** Make escribe filas en el rango A–Q. Una fórmula "spill" (ARRAYFORMULA) dentro de ese rango se rompe cuando Make mete una fila. Por eso las versiones vivas van en R y S, fuera del alcance de Make.

Cómo se llena cada fila

Filas históricas (2018–2026, importadas)

```
pelis_consolidado (registro crudo del usuario)
  → peliculas_import.csv  (2018-2025)  +  pelis_2026.csv  (2026, verificado a mano)
  → build_catalogo.js  →  catalogo_completo.csv  →  importado a la hoja
```

Filas nuevas (bot)

```
Telegram "vi X"
  → Gemini extrae C,D,E,F,L,N
  → Make agrega la fila: G = hoy, H = año actual, J = "Enriquecida IA",
    K = "Bot Telegram", A/B/I por fórmula
```

Datos enriquecidos (fuera de la hoja: TMDB y machine learning)

Viven en la carpeta `data/` del repo, no en la hoja — todos con la misma clave: `normalizar(titulo) + "|" + anio_estreno`. Cubren el catálogo y la lista "Quiero ver".

Archivo	Contenido	Script que lo genera	Cobertura
`posters.json`	`{ poster, rating, genres, tmdb }`	`cerebro/build_posters.js`	1.263 con póster de 1.275 (11 sin match en TMDB)
`actors.json`	reparto principal (los 8 primeros)	`cerebro/build_actors.js`	1.264
`runtime.json`	duración en minutos	`cerebro/build_runtime.js`	1.264
`synopsis.json`	sinopsis en castellano (TMDB)	`cerebro/build_synopsis.js`	1.240

Archivo	Contenido	Script que lo genera	Cobertura
`taste_profile.json`	perfil de gusto: qué predice que una película se vuelva a ver (Random Forest + KMeans, `random_state` fijo)	`cerebro/taste_profile.py`	—
`similar.json`	5 parecidas por contenido (género, director, reparto, década + TF-IDF de sinopsis)	`cerebro/build_similar.py`	1.238
`similar_ajustado.json`	las mismas candidatas, reponderadas con el perfil de gusto	`cerebro/build_similar_ajustado.py`	1.238
`directors.json`	directores de cada película con su género según TMDB (`f` / `m` / `nb` / `?`)	`cerebro/build_directors.js`	1.265
`genero-reparto.json`	género de cada persona del reparto principal, según TMDB	`cerebro/build_cast_gender.js`	~6.800 personas
`genero-manual.json`	correcciones a mano donde TMDB no tiene el género cargado	—	(a mano)
`posters-manual.json` + `img/`	las 10 que TMDB no tiene o matchea mal, cargadas a mano	—	10
`mis9.json`	la selección fija de "Mis 9 películas" del dueño	`cerebro/set_mis9.js`	(opcional)

Cómo se mantienen: la sincronización diaria (`.github/workflows/sync-datos.yml`, GitHub Actions, 07:00) corre todos los scripts en ese orden y commitea solo si algo cambió. Los scripts de TMDB leen el `tmdb` id ya resuelto en `posters.json` (nunca vuelven a buscar por título) y saltan las claves que ya están pobladas, así que solo trabajan sobre lo nuevo — para regenerar algo de cero hay que borrar ese archivo primero. Los de Python recalculan todo cada vez, pero con semilla fija: con los mismos datos, el mismo resultado (y por lo tanto ningún commit).

Si una película se agrega después de la sincronización (ej. a la tarde), el sitio busca su póster en TMDB en el momento, hasta que la corrida del día siguiente la deje guardada.

Los matches errados de TMDB (remakes y homónimos) se corrigieron con scripts de una sola vez que eligen por director en vez de por popularidad; ya no están en el repo (siguen en el historial de git) y sus correcciones quedaron en `posters.json`, que la sincronización no pisa. `cerebro/check_datos.js` sigue disponible para detectar casos nuevos.

El sitio mergea `posters-manual.json` **encima** de `posters.json`, así que regenerar el backfill no pisa lo cargado a mano. Las rutas de `img/` son relativas a la raíz del repo y el sitio las resuelve contra GitHub.

Visionados vs. películas

La hoja tiene una fila por **visionado**: ~1.310 filas corresponden a ~1.170 **películas distintas** (las revisitadas suman varias filas). Los gráficos de la sección Datos del sitio tienen un selector "Sin repetir / Contando repetidas" para elegir cuál de las dos cosas contar. En los gráficos por fecha (por año, por mes), "sin repetir" cuenta cada película una vez por período.

Gotcha: corregir el `anio_estreno` de una fila en la hoja **huérfana** la clave vieja en estos JSON (`título|2023` → `título|2025`) — hay que re-buscar o re-mapear la clave a mano, si no el póster/reparto/etc. quedan sin datos aunque el título esté bien escrito.

`anio_visto` vs `fecha_vista` (resuelto)

`anio_visto` llegó a tener el año **corrido** respecto a `fecha_vista` en 8 bloques de la planilla histórica + 4 filas sueltas (la lista "2018" tenía fechas de 2019; la "2021", de 2022; la "2025", de 2026) — arrastre de cómo se armaban las pestañas anuales originales. Se corrigió con una fórmula en columna auxiliar + pegado especial (solo valores) sobre `fecha_vista`, y `anio_visto` se recalculó a partir de eso. Verificado contra las 1.288 filas del CSV publicado: 0 mismatches. El gráfico "por año" del sitio usa `anio_visto`, ya consistente.

Acceso

- **Lectura pública:** el CSV publicado (`…/pub?gid=…&single=true&output=csv`) — es lo que consumen el sitio, CinefilIA y la sincronización diaria.
- **Escritura:** solo el usuario (a mano) y Make (vía su conexión OAuth de Google).
- **Backup:** `cerebro/catalogo_completo.csv`, que la sincronización diaria actualiza con la hoja publicada (si Google falla, no pisa el anterior) — así el historial de git es también un historial de la planilla. Además, `cerebro/Pelis_backup_*.xlsx` (con las pestañas por año, local, ignorado por git).